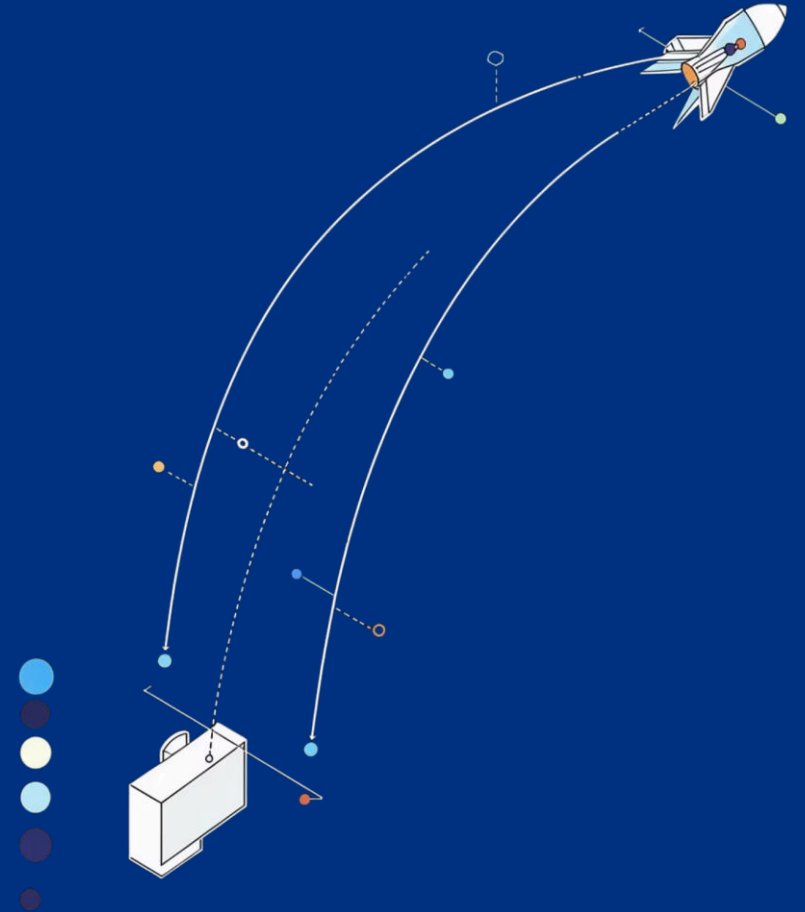


# Simulación de Tiro Parabólico: Ideal vs Real

Proyecto Final Integrador — Física I — Ingeniería en  
Informática



# ¿Por qué este proyecto?

El mundo ideal no existe

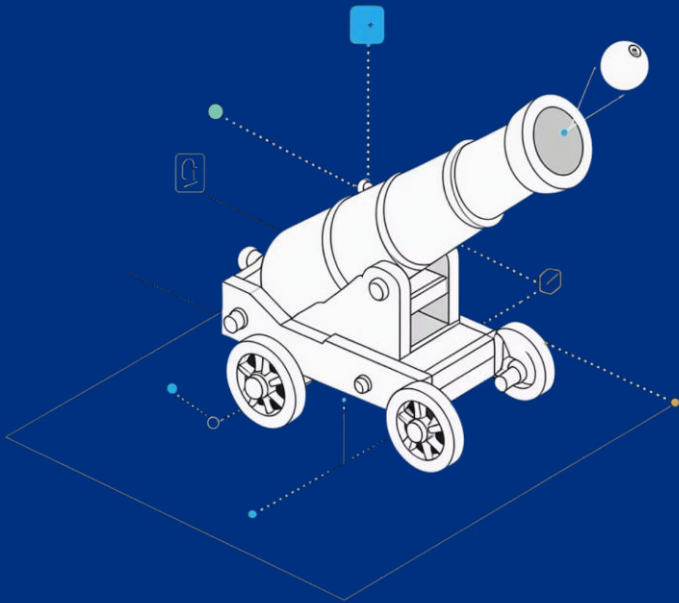
El tiro parabólico es un clásico de la Física, pero los libros asumen un mundo sin aire.

La realidad es diferente

La resistencia aerodinámica cambia todo: alcance, altura máxima y velocidad de impacto.

Nuestra propuesta

Modelar ambos escenarios y comparar los resultados con precisión numérica.



# Dos modelos, dos realidades

## Modelo Ideal

Solo gravedad, sin rozamiento. Fórmulas simples, parábola perfecta y solución analítica cerrada.

## Modelo Real

Resistencia del aire cuadrática. Ecuaciones acopladas en X e Y, sin solución analítica posible.

Ambos parten del mismo lanzamiento — pero llegan a resultados radicalmente distintos.



# Física del modelo ideal

En el modelo ideal, la única fuerza actuante es la gravedad. Esto genera ecuaciones sencillas y una trayectoria perfectamente simétrica.

- Aceleración horizontal = 0  $\rightarrow$  velocidad horizontal constante
- Aceleración vertical =  $-g \rightarrow$  caída gravitatoria uniforme
- Resultado: parábola simétrica, alcance máximo exactamente a  $45^\circ$

# Física del modelo real

## Fuerza de arrastre

La resistencia aerodinámica depende del cuadrado de la velocidad:

$$F_d = \frac{1}{2} \rho C_d A v^2$$

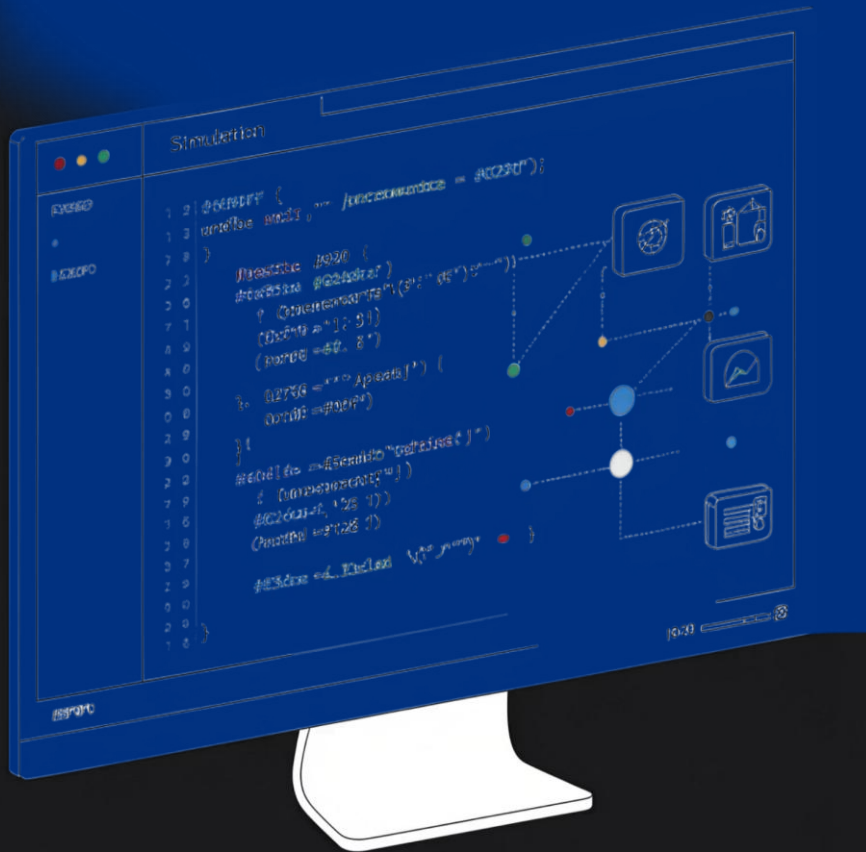
Donde  $\rho$  es la densidad del aire,  $C_d$  el coeficiente de arrastre y  $A$  el área frontal del proyectil.

## El acoplamiento cambia todo

Ambos ejes quedan ligados: la velocidad total  $v$  afecta simultáneamente a las componentes horizontal y vertical.

❏ No existe fórmula cerrada → necesitamos métodos numéricos.

# Métodos numéricos: Euler vs RK4



## Método de Euler

Rápido de implementar pero acumula errores en cada paso. Aproximación de **primer orden** — válido para estimaciones gruesas.

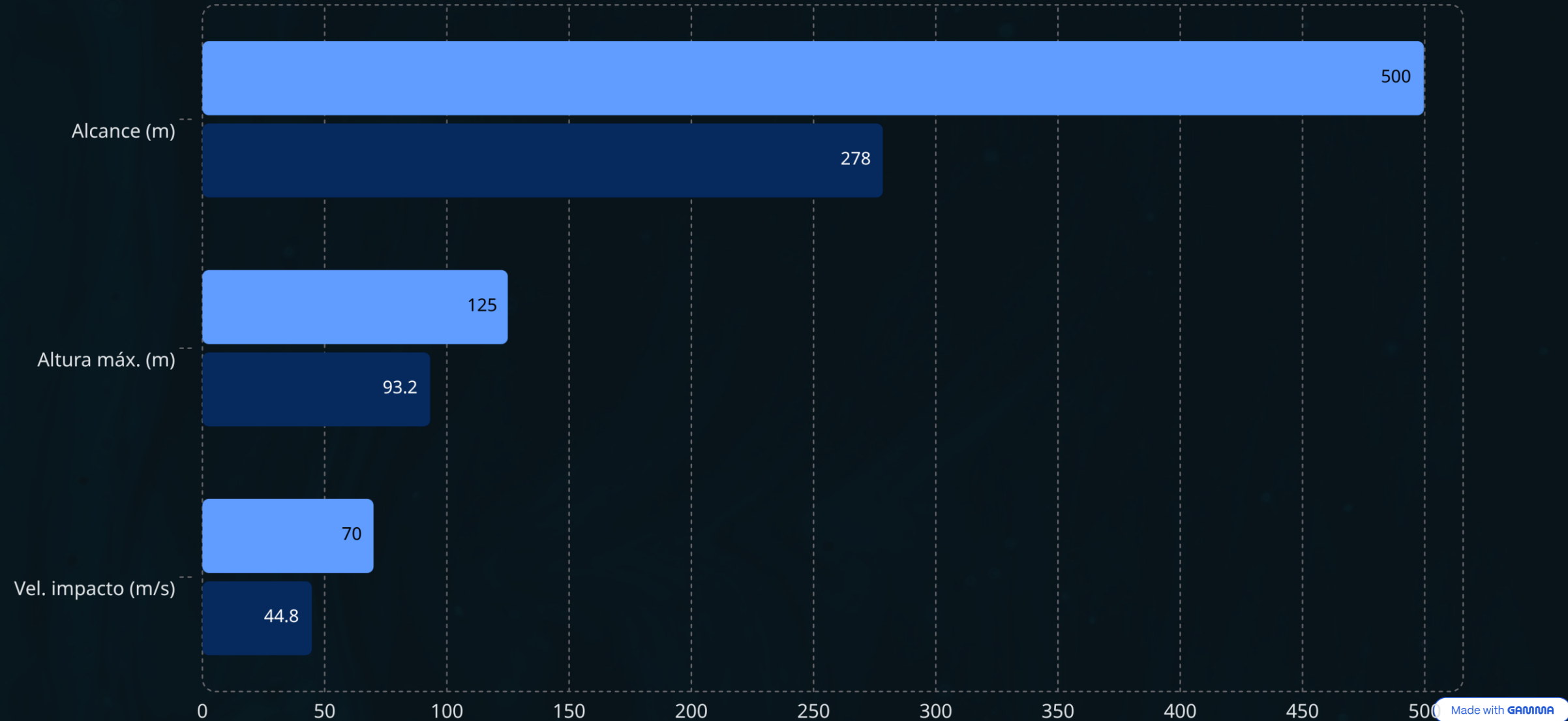
## Runge-Kutta 4° Orden (RK4)

Cuatro evaluaciones por paso, error de **cuarto orden**. Preciso, estable y robusto. Es el método que implementamos en nuestro simulador.

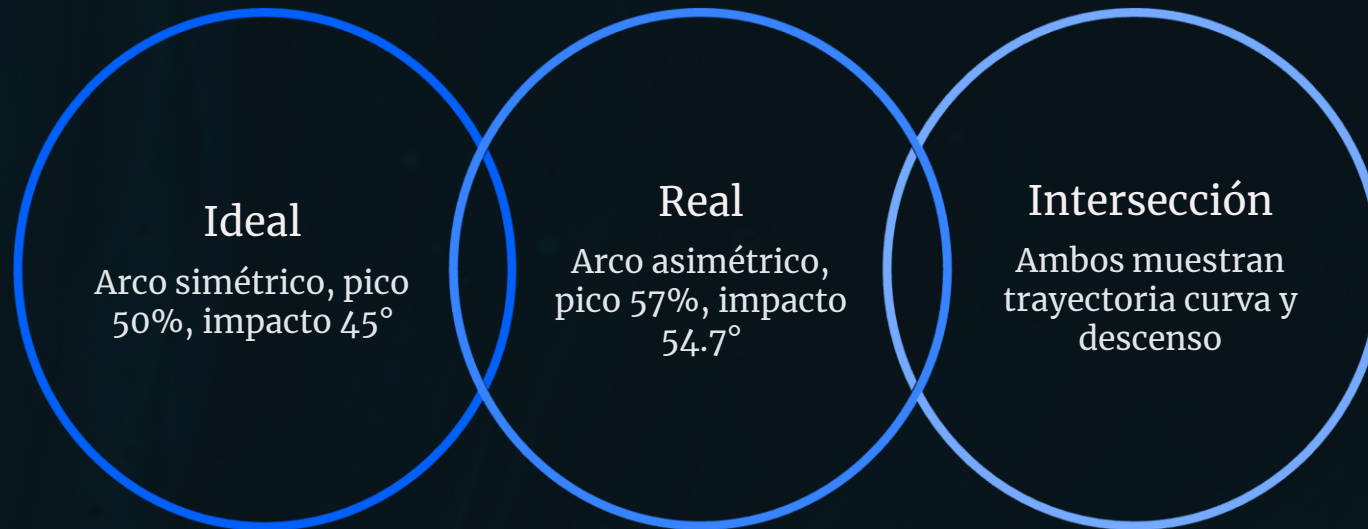
# Resultados: El impacto del aire

■ Ideal ■ Real

Métrica



# Pérdida de simetría



## La curva pierde su equilibrio

- **Ideal:** punto más alto exactamente a mitad de camino (50% del alcance)
- **Real:** el pico ocurre aproximadamente antes (57%) del alcance, ligeramente después de la mitad del recorrido y la caída es notablemente más empinada

## Ángulo de impacto

- **Ideal:** 45° — simétrico al ángulo de lanzamiento
- **Real:** 54.7° — el impacto es más vertical, Aunque su velocidad es menor

# Arquitectura del código



## modelo.py

Define los parámetros físicos y las ecuaciones de ambos modelos usando Programación Orientada a Objetos.



## simulador.py

Implementa el integrador RK4 y ejecuta la simulación paso a paso en el tiempo.



## grafica.py

Genera el gráfico estático comparative de la trayectoria.



## animacion.py

Produce una animación interactiva en tiempo real de ambas trayectorias simultáneas.



## main.py

Punto de entrada que orchestra todos los módulos y gestiona los parámetros de entrada.



## menu.py

Despliega la Interfaz interactiva por consola para modificar variables.

# Conclusión

La resistencia del aire no es un detalle

Reduce el alcance un **44%**, la altura un **25%** y la velocidad de impacto un **36%**.

Ignorarla no es una simplificación aceptable en aplicaciones reales.

RK4: precisión donde las ecuaciones fallan

El método de Runge-Kutta de 4° orden nos permitió resolver ecuaciones no lineales acopladas con alta estabilidad y sin solución analítica.

La herramienta hace visible lo invisible

La simulación computacional convierte la cinemática abstracta en movimiento real e interactivo — eso es ingeniería aplicada a la física.